

BUILD VARIANT A

Variant A - Off-the-Shelf Chassis

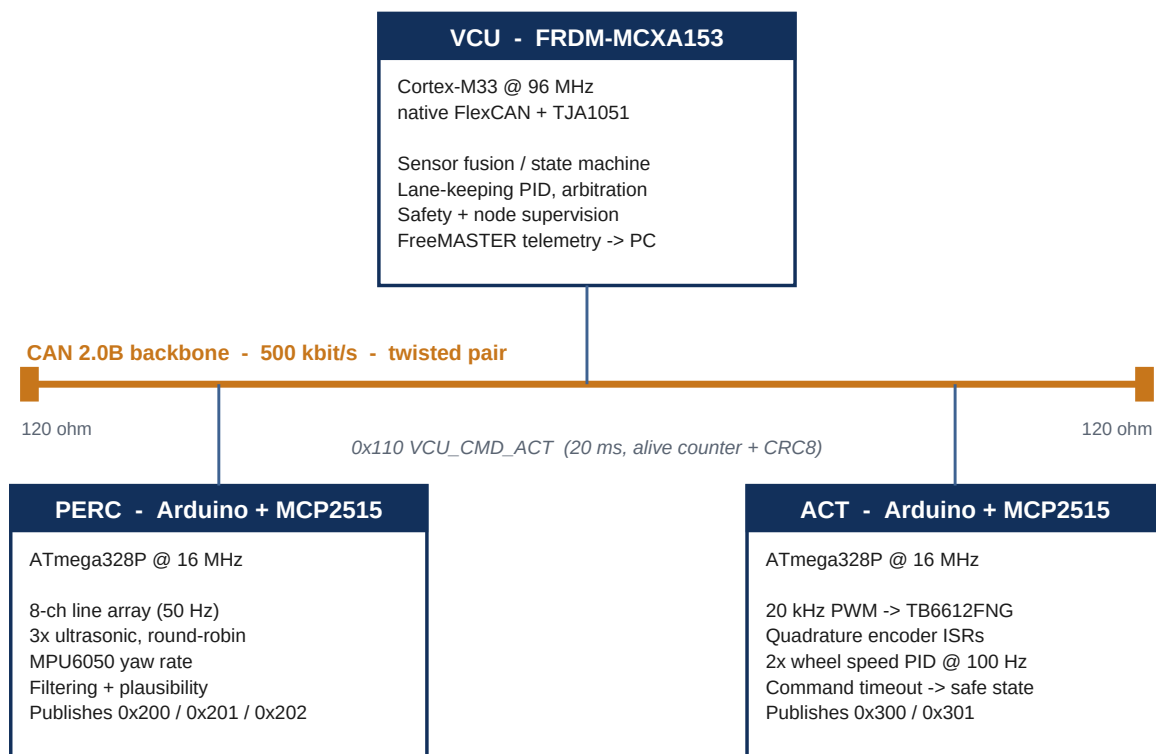
Three-ECU CAN autonomous vehicle built on a commercial 4WD chassis kit. Fastest route to a working demonstrator; all schedule risk sits in firmware.

1. System overview

A small autonomous vehicle built as **three independent electronic control units** communicating over a single classic CAN bus. The vehicle performs lane keeping (line following), obstacle detection and emergency braking. The target demonstration is not the driving itself but **fault-injection resilience**: a node is physically disconnected mid-lap and the vehicle degrades safely instead of crashing.

Why distribute at all? At this scale one MCXA153 could run everything - it sits at roughly 15 % load. The genuine justifications are timing isolation (ultrasonic ranging blocks for up to 30 ms and must stay away from a 100 Hz control loop), fault containment, I/O locality, and scalability: a fourth node costs one CAN stub, not a redesign. State this honestly in the presentation - it is a stronger answer than pretending CPU is the bottleneck.

2. Node architecture



Estimated bus load: 200 frames/s x ~130 bit = ~26 kbit/s = 5.2 %

Bus constraint. The MCP2515 is **classic CAN 2.0B only - it has no CAN-FD support**. The whole bus therefore runs classic CAN at 500 kbit/s and the MCXA153 FlexCAN is configured in non-FD mode, even though the silicon supports FD. CAN-FD is the version-2 upgrade path once the Arduino nodes are replaced.

3. Electronics and networking - bill of materials

Prices are indicative single-unit EU retail, excluding shipping. This section is **identical in both build variants**.

3.1 Compute and CAN networking

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
1	FRDM-MCXA153 development board	1	VCU node - Cortex-M33, native FlexCAN	17.00	17.00	Have
2	Arduino UNO R3 / Nano	2	PERC and ACT nodes	12.00	24.00	Have
3	TJA1051T/3 or SN65HVD230 transceiver breakout	1	CAN PHY for the MCX native FlexCAN	4.00	4.00	Buy
4	MCP2515 + TJA1050 CAN module	2	SPI CAN controller - AVR has no CAN core	4.00	8.00	Buy
5	120 ohm 0.25 W resistor	2	Bus termination, two physical ends only	0.10	0.20	Buy
6	USB-CAN analyzer (CANable / USBtin) + SavvyCAN	1	Bus sniffing, DBC decode, logging, fault injection	18.00	18.00	Buy

3.2 Sensors

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
7	QTR-8A reflectance array (or 5x TCRT5000)	1	Line / lane position, analog	10.00	10.00	Buy
8	HC-SR04 ultrasonic (or VL53L0X ToF)	3	Obstacle distance left / centre / right	2.00	6.00	Buy
9	MPU6050 IMU	1	Yaw rate and tilt, I2C	3.00	3.00	Buy

3.3 Power and protection

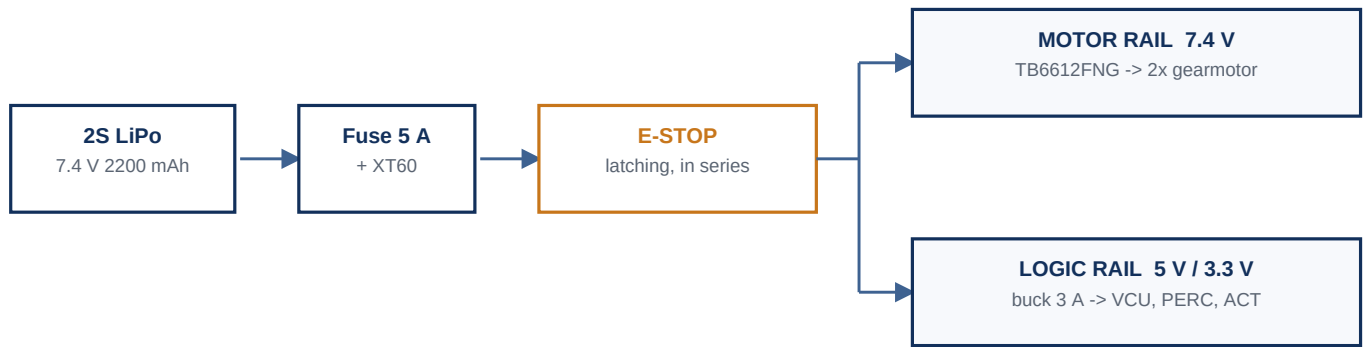
#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
10	LiPo 2S 7.4 V 2200 mAh + balance charger	1	Traction and logic supply	25.00	25.00	Buy
11	Buck converter 5 V / 3 A (MP1584 or LM2596)	1	Logic rail, isolated from motor rail	4.00	4.00	Buy
12	Latching emergency-stop button, 10 A	1	Hardware safe state, firmware-independent	5.00	5.00	Buy
13	Bulk cap 470-1000 uF 25 V + 100 nF assortment	1	Motor rail decoupling	3.00	3.00	Buy
14	XT60 connectors + fuse holder + 5 A fuse	1	Battery interface and overcurrent protection	4.00	4.00	Buy

3.4 Wiring and prototyping

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
15	JST-XH / JST-GH connector kit + crimp tool	1	Keyed ECU harness - CAN_H, CAN_L, VBAT, GND	12.00	12.00	Buy
16	Protoboard 2x, pin headers, jumper wire	1	Node interconnect and transceiver carriers	8.00	8.00	Buy
17	Silicone wire 22 AWG, heat shrink, cable ties	1	Harness build	6.00	6.00	Buy

Electronics subtotal (items to purchase): EUR 116.20 - the two boards already owned (EUR 41.00 of value) are excluded.

3.5 Power architecture



Single-point (star) ground between motor and logic returns. Bulk 470-1000 uF across the motor rail. E-stop removes motor energy independently of all firmware.

4. Mechanical structure - commercial chassis kit

The standard acrylic 4WD kit (two-deck plate, four TT gearmotors, wheels and hardware) is used as delivered. Two motors are driven and the other two run free, or all four are paired left/right - a decision to make on the bench.

4.1 Mechanical bill of materials

#	Component	Qty	Role in the system	Unit EUR	Total EUR	Status
A1	4WD chassis kit - acrylic plate, 4x TT motor, wheels, hardware	1	Complete rolling platform	18.00	18.00	Buy
A2	TB6612FNG dual motor driver breakout	1	Motor bridge - 1.5 V lower dropout than an L298N	4.00	4.00	Buy
A3	Slotted optical encoder + encoder disc	2	Wheel speed feedback for the ACT node PID	4.00	8.00	Buy
A4	M3 nylon standoff, screw and nut assortment	1	Board and deck mounting	6.00	6.00	Buy
A5	Aluminium / acrylic bracket for the line sensor	1	Holds the array 3-8 mm above the floor	3.00	3.00	Buy
Mechanical subtotal					39.00	

4.2 What you gain and what you give up

Advantage	Limitation
Rolling platform in about one hour - the whole schedule goes to firmware	TT gearmotors have a sloppy output shaft; encoder discs wobble and odometry is noisy
Known-good geometry, no tolerance work, no printer dependency	Fixed hole pattern - moving a board or sensor means drilling acrylic
Cheapest variant, and every part is instantly replaceable	No mechanical intellectual property is owned, which weakens the productisation story
Zero calendar risk from fabrication	Encoders are retrofitted, not integrated - alignment is fiddly and drifts

4.3 Physical layout rules

- **Lower deck = power domain:** battery, buck converter, TB6612FNG, motors. **Upper deck = logic domain:** the three ECU boards and the CAN daisy chain. The mechanical split mirrors the electrical one.
- Run the CAN bus as a **single linear backbone** past all three nodes, terminated only at the two physical ends. No star topology, no long stubs.
- Mount the line-sensor bracket **60-100 mm ahead of the drive axle**. This lookahead distance directly sets PID behaviour: too short oscillates, too long cuts corners. Use slotted holes so it can be tuned mechanically.
- Keep motor wiring short, twisted and away from the sensor harness and the CAN pair.
- Fit the latching E-stop where it can be hit while the vehicle is moving.

4.4 Ten-day build schedule

Day	Milestone
1	Toolchains installed. Blink and UART running on all three boards. Do not skip this.
2	Two-node CAN loopback: MCX to analyzer, then MCX to Arduino. Nothing else until frames flow.
3	All three nodes on the bus, periodic status frames, .dbc decoding in SavvyCAN.
4	Chassis kit assembled. Motors, driver and encoders wired. ACT drives open-loop from a CAN command.
5	ACT closed-loop speed PID, tuned live through FreeMASTER on the VCU.
6	PERC publishing line and distance. GATE: the vehicle follows a line. Minimum demo secured.
7	Obstacle detection, emergency braking, full vehicle state machine.

Day	Milestone
8	E2E alive counter and CRC8, timeout supervision, degraded mode.
9	Fault-injection test campaign. FreeRTOS port only if the schedule allows.
10	Poster, video, demo rehearsal.

If the school runs only one week, stop at the day-6 gate and spend the remaining time on safety and fault injection. A reliable line follower with real fault handling beats an unreliable everything.

5. Communication structure - CAN message set

Classic CAN 2.0B, 500 kbit/s, standard 11-bit identifiers. **Lower identifier = higher priority.** Design and freeze this table before writing any application code, and express it as a real **.dbc** file so the analyzer decodes named signals live.

ID	Message	Producer	Period	DLC	Payload
0x080	EMERGENCY_STOP	any node	event	1	reason code - highest priority on the bus
0x100	VCU_STATUS	VCU	100 ms	2	vehicle state, alive counter
0x110	VCU_CMD_ACT	VCU	20 ms	8	speed_sp i16, yaw_rate_sp i16, mode, alive_ctr, crc8
0x200	PERC_DIST	PERC	50 ms	6	dist_left / dist_centre / dist_right, u16 mm
0x201	PERC_LINE	PERC	20 ms	4	line_pos i16 (-1000..+1000), valid_flags
0x202	PERC_STATUS	PERC	100 ms	3	alive_ctr, per-sensor health bits
0x300	ACT_FEEDBACK	ACT	20 ms	8	speed_meas_L, speed_meas_R, odometry ticks
0x301	ACT_STATUS	ACT	100 ms	3	alive_ctr, fault bits, driver temperature flag

Bus load. About 200 frames/s x ~130 bit worst case = ~26 kbit/s = **5.2 % of 500 kbit/s**. Automotive practice keeps a bus below 40-50 %, so there is ample headroom for a fourth node. Put this calculation on the poster - it shows the bus was sized, not guessed.

6. Safety concept

A miniature functional-safety argument. Safe state is defined as: **motors de-energised, brake applied, hazard LEDs on, buzzer active.**

Hazard / fault	Mechanism
Corrupt or stale command	E2E profile on 0x110: alive counter + CRC8; ACT rejects and counts violations
Node silent / crashed	Timeout supervision on every node. ACT: no valid command for 100 ms -> safe state
Perception lost	VCU: no PERC frame for 150 ms -> DEGRADED, speed capped at 30 %
Firmware hang	Watchdog on all three (WDOG on MCX, WDT on AVR)
Any hazard, operator-triggered	Latching hardware E-stop in the motor supply, independent of firmware
Implausible sensor reading	Range and rate-of-change checks; sensor marked invalid in health bits
Bus flooded / rogue frame	0x080 EMERGENCY_STOP holds the lowest ID, so it always wins arbitration

6.1 Fault-injection test plan

This is the demonstration that distinguishes the project from a hobby robot. Rehearse every row.

Injected fault	Expected observable behaviour
Unplug PERC mid-lap	VCU enters DEGRADED within 150 ms, speed capped, vehicle keeps lane
Short CAN_H to CAN_L	All nodes lose comms, ACT times out, motors de-energised within 100 ms
Remove a termination resistor	Increased error frames visible in the analyzer; log and demonstrate
Brown out the 5 V rail	Watchdog reset, node re-enters INIT and re-joins the bus
Inject a rogue frame from the analyzer	E2E check fails, frame discarded, violation counter increments
Corrupt the CRC deliberately	ACT holds last valid command briefly then falls to safe state

7. Software structure

The VCU is built with the MCUXpresso SDK in C, starting from the *flexcan_interrupt_transfer* example for the board. Use **MCUXpresso Config Tools** to mux CAN0_TX / CAN0_RX onto header pins and to compute bit timing - do not calculate the segments by hand. Begin with a 1 ms time-triggered scheduler rather than an RTOS: it is deterministic, trivially explainable, and can be ported to FreeRTOS late if time allows.

Use NXP FreeMASTER. It provides live variable scoping and on-the-fly PID tuning over the MCU-Link VCOM. Tuning a PID by edit-and-reflash will consume the entire schedule; FreeMASTER turns it into a slider.

Node	Module	Responsibility
VCU	can_if.c	FlexCAN init (classic, 500k), RX filters, TX mailboxes
	pdu_router.c	Signal pack/unpack, E2E alive counter + CRC8 verify
	fusion.c	Distance/line filtering, obstacle classification
	vehicle_sm.c	INIT / SELF_TEST / IDLE / ARMED / DRIVE / DEGRADED / SAFE_STOP
	path_ctrl.c	Lane-keeping PID (line_pos to yaw_rate) + speed profile
	safety_mon.c	Per-node timeout table, degraded and safe-state logic
	telemetry.c	LPUART to FreeMASTER over the MCU-Link VCOM
	sched.c	1 ms SysTick time-triggered task table
PERC	us_driver.cpp	Non-blocking ultrasonic FSM, micros() + pin-change ISR
	line_array.cpp	8-channel ADC sweep, calibration, centroid to line_pos
	filters.cpp	Median-of-3 + IIR, range plausibility, health bits
	can_node.cpp	MCP2515 driver wrapper, periodic TX scheduler
ACT	motor.cpp	Timer1 20 kHz PWM, TB6612FNG direction logic, slew limit
	encoder.cpp	Quadrature decode ISRs, tick accumulation, speed estimate
	speed_pid.cpp	Per-wheel PI(D) @ 100 Hz, anti-windup
	kinematics.cpp	(speed, yaw_rate) to per-wheel setpoints, odometry
	safe_state.cpp	Command timeout, fault latch, coast/brake sequence
shared	veh_can_ids.h	Message IDs, signal offsets, scaling - identical on all 3
	veh_geometry.h	Wheel diameter, track width, ticks-per-rev, mm-per-tick
	e2e_crc8.c/.h	CRC8 and alive-counter helpers, host-testable

8. Computational load distribution

Node	Core	Task	Rate	Load
VCU	Cortex-M33 @ 96 MHz	CAN RX/TX, E2E check, signal unpack	event	3 %
		Sensor fusion and filtering	50 Hz	3 %
		Lane-keeping PID + speed profile	50 Hz	2 %
		State machine + node supervision	100 Hz	2 %
		FreeMASTER / UART telemetry	20 Hz	5 %
		VCU total		~15 %
PERC	ATmega328P @ 16 MHz	Ultrasonic round-robin (non-blocking FSM)	20 Hz	5 %
		8-channel ADC line sweep (~0.9 ms)	50 Hz	5 %
		Filtering, plausibility, SPI/CAN TX	50 Hz	10 %
		PERC total		~20 %
ACT	ATmega328P @ 16 MHz	Quadrature encoder ISRs	~4.8 kHz	3 %

Node	Core	Task	Rate	Load
		2x wheel speed PID	100 Hz	2 %
		CAN RX + timeout supervision	50 Hz	6 %
		ACT total		~12 %

Headroom on the VCU is deliberate - it is where stretch goals live (Kalman filter, occupancy grid, Stanley controller).

9. Risk register

Risk	Severity	Mitigation
MCP2515 crystal mismatch	High	Modules ship with 8 MHz or 16 MHz crystals. Wrong setting = zero frames and no error. Read the crystal marking before wiring.
Triple bus termination	High	MCP2515 modules carry an onboard 120 ohm. Three nodes = 40 ohm = dead bus. Remove it on the middle node.
Motor-induced brownout	High	Separate rails, star ground, bulk capacitance, flyback protection. Most common cause of unexplained resets.
Blocking ultrasonic reads	Medium	pulseIn() stalls up to 30 ms. Must be interrupt-driven from day one.
Loose Dupont wiring	Medium	Intermittent faults that look exactly like CAN bugs. Crimp a proper harness.
Scope creep (camera vision)	Medium	Not feasible on these MCUs. If wanted, add an ESP32-CAM as a fourth node publishing lane offset - a stretch goal, never a dependency.

10. Path to a viable product at small scale

The vehicle is not the product - the teaching platform is: a three-ECU automotive networking lab kit for universities, hackerspaces and corporate embedded training. On this variant the electronics are productisable but the mechanics are not, because the chassis is somebody else's product being resold.

Step	Action	Why it matters
1	Replace both Arduinos with MCX A14x nodes	One toolchain, one codebase, native CAN-FD. About EUR 1-2 per MCU at volume against EUR 12 per Arduino
2	One generic ECU node PCB in two stuffing variants (PERC / ACT)	Two-layer board at roughly EUR 2 each in quantity 10; replaces breadboard wiring
3	Keyed JST-GH 4-pin daisy chain (CAN_H, CAN_L, VBAT, GND)	Removes the single largest source of intermittent field faults
4	CAN bootloader on every node	Field firmware update over the bus - the biggest single value-add, and a great feature to build
5	End-of-line test fixture: pogo-pin jig that flashes, self-tests and stamps a serial	Required the moment more than about ten units are built
6	Curriculum package: lab manual, .dbc, reference firmware, exercise set	The margin is in the curriculum, not the hardware
7	Source or commission a chassis rather than reselling a hobby kit	The kit is the weak link commercially - no mechanical IP is owned, and supply is uncontrolled

Regulatory gates for EU sale. Sold as a kit, the assembled product falls under the EMC Directive. **Ship without the LiPo** to avoid UN38.3 transport testing - standard practice for drone and robot kits. License the hardware CERN-OHL-S and the firmware Apache-2.0: community adoption is the marketing channel for a platform like this.

Total project cost, Variant A: EUR 155.20 to purchase (electronics EUR 116.20 + mechanics EUR 39.00), on top of the FRDM-MCXA153 and two Arduinos already owned.